Practical Session 3: Mastering Conditionals in Java

Student-Centered, Self-Paced Lab

➢ Learning Outcomes Covered

By the end of this practical session, you will be able to:

1. **Design** a decision structure that handles multiple conditions using nested if-else or switch appropriately.
2. **Analyze** complex boolean expressions to determine all possible execution paths.
3. **Evaluate** the readability and efficiency of different conditional structures for the same logic.
4. **Debug** a faulty conditional statement by tracing execution with various test inputs.

Time Required: 90–120 minutes
Instructor Intervention: None – you will figure it out yourself!

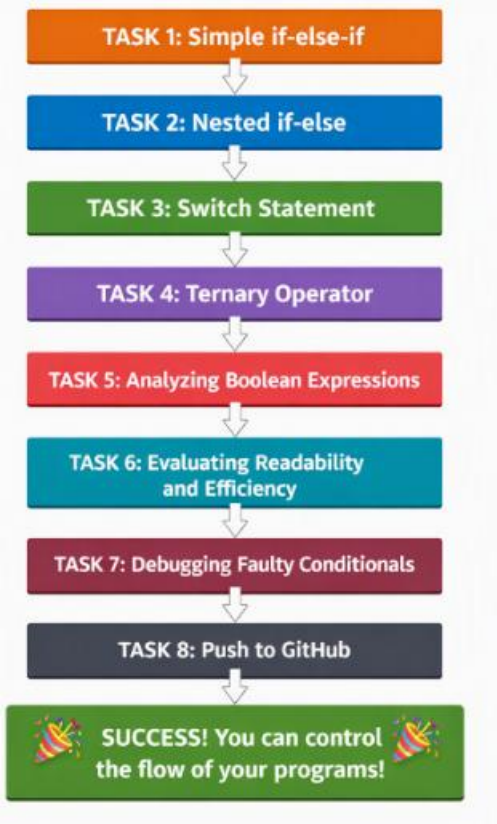
➢ Before You Begin

Ensure you have completed **Week 1 and Week 2 practical sessions**. You will need:

- VS Code open
- Your java-course folder with week2 subfolder
- Git configured and connected to GitHub

- **Your Roadmap**

PRACTICAL SESSION 3 ROADMAP



TASK 1: Simple if-else-if

Objective

Write a program that checks if a number is positive, negative, or zero.

Step 1.1: Create a New Folder

1. Open **VS Code**.
2. Open your java-course folder.
3. Right-click the java-course folder → **New Folder**.
4. Name it: week3
5. Press Enter.

Step 1.2: Create the File

1. Right-click the week3 folder → **New File**.
2. Name the file: PositiveNegativeZero.java
3. Press Enter.

Step 1.3: Write the Code

Copy and paste this code into PositiveNegativeZero.java:

java

```
// This program checks if a number is positive, negative, or zero.


public class PositiveNegativeZero {
   public static void main(String[] args) {


      // Declare an integer variable named number and assign it the value 10.
      int number = 10;


      // Check if the number is greater than 0.
      if (number > 0) {
         // If true, print that the number is positive.
         System.out.println(number + " is positive.");
      }
      // If the first condition is false, check if the number is less than 0.
      else if (number < 0) {
         // If true, print that the number is negative.
         System.out.println(number + " is negative.");
      }
      // If both conditions are false, the number must be zero.
```

```
        else {
            // Print that the number is zero.
            System.out.println(number + " is zero.");
        }
    }
}
```

Step 1.4: Run the Program

1. Press Ctrl + S to save.
2. Click the **Run** button (▶️) at the top-right of VS Code.
3. Observe the output: 10 is positive.

Step 1.5: Experiment

Change the value of number to:

- -5 → What happens?
- 0 → What happens?

✅ **Task 1 Complete**

- □ I created PositiveNegativeZero.java
- □ I ran the program successfully
- □ I experimented with different values

TASK 2: Nested if-else

Objective

Write a program that determines a student's grade based on marks and also checks if the marks are valid.

Step 2.1: Create the File

Create a new file called StudentGrade.java in the week3 folder.

Step 2.2: Write the Code

java

```
// This program determines a student's grade based on marks.
// It uses nested if-else to first validate the marks, then assign a grade.

public class StudentGrade {
    public static void main(String[] args) {

        // Declare an integer variable for marks and assign it the value 75.
        int marks = 75;

        // Outer if: check if marks are within the valid range (0 to 100).
        if (marks >= 0 && marks <= 100) {
            // Marks are valid. Now determine the grade using inner if-else.

            // Check if marks are 70 or above.
            if (marks >= 70) {
                // If true, print grade A.
                System.out.println("Grade: A");
            }
            // If marks are not >= 70, check if they are >= 60.
            else if (marks >= 60) {
                // If true, print grade B.
                System.out.println("Grade: B");
            }
            // If marks are not >= 60, check if they are >= 50.
```

```
        else if (marks >= 50) {
            // If true, print grade C.
            System.out.println("Grade: C");
        }
        // If marks are not >= 50, check if they are >= 40.
        else if (marks >= 40) {
            // If true, print grade D.
            System.out.println("Grade: D");
        }
        // If none of the above, marks must be below 40.
        else {
            // Print grade F (Fail).
            System.out.println("Grade: F (Fail)");
        }
    }
    // If marks are not between 0 and 100, they are invalid.
    else {
        // Print an error message.
        System.out.println("Invalid marks! Please enter a value between 0 and 100.");
    }
  }
}
```

Step 2.3: Run the Program

Save and run. Expected output: Grade: B

Step 2.4: Experiment

Change marks to:

- -10 → What happens?
- 105 → What happens?
- 45 → What grade?
- 100 → What grade?

✅ **Task 2 Complete**

- □ I created StudentGrade.java
- □ I ran the program successfully
- □ I experimented with different marks

TASK 3: Switch Statement

Objective

Write a menu-driven program for an M-Pesa agent using a switch statement.

Step 3.1: Create the File

Create a new file called MpesaMenu.java in the week3 folder.

Step 3.2: Write the Code

java

```
// This program simulates a simple M-Pesa agent menu using a switch statement.


import java.util.Scanner; // Import the Scanner class for user input.


public class MpesaMenu {
    public static void main(String[] args) {


        // Create a Scanner object to read input from the keyboard.
        Scanner scanner = new Scanner(System.in);
```

```
// Display the menu options.
System.out.println("=== M-PESA AGENT MENU ===");
System.out.println("1. Check Balance");
System.out.println("2. Send Money");
System.out.println("3. Withdraw Cash");
System.out.println("4. Exit");
System.out.print("Enter your choice (1-4): ");

// Read the user's choice as an integer.
int choice = scanner.nextInt();

// Use a switch statement to handle the choice.
switch (choice) {
   // Case 1: Check Balance
   case 1:
      // Print a message for balance inquiry.
      System.out.println("Your balance is KSh 10,000.00");
      // Break out of the switch to prevent fall-through.
      break;

   // Case 2: Send Money
   case 2:
      // Print a message for sending money.
      System.out.println("Send Money selected.");
      // Break out of the switch.
      break;
```

```
        // Case 3: Withdraw Cash
        case 3:
            // Print a message for withdrawal.
            System.out.println("Withdraw Cash selected.");
            // Break out of the switch.
            break;

        // Case 4: Exit
        case 4:
            // Print a goodbye message.
            System.out.println("Thank you for using M-Pesa. Goodbye!");
            // Break out of the switch.
            break;

        // Default case: handles any input not matching 1-4.
        default:
            // Print an error message for invalid choice.
            System.out.println("Invalid choice. Please enter a number between 1 and 4.");
            // Break is optional here but good practice.
            break;
    }

    // Close the scanner to free resources.
    scanner.close();
}
```

}

Step 3.3: Run the Program

Save and run. Try each option (1, 2, 3, 4, and an invalid number like 5).

Step 3.4: Experiment

Remove one break statement. What happens? Why?

✅ **Task 3 Complete**

- □ I created MpesaMenu.java
- □ I ran the program with different choices
- □ I understand the purpose of break

TASK 4: Ternary Operator

Objective

Rewrite a simple if-else using the ternary operator.

Step 4.1: Create the File

Create a new file called TernaryDemo.java in the week3 folder.

Step 4.2: Write the Code

java

```
// This program demonstrates the ternary operator as a shorthand for if-else.


public class TernaryDemo {
    public static void main(String[] args) {


        // Declare an integer variable for age.
        int age = 20;


        // Using if-else (traditional way)
```

```
        String eligibilityIfElse;
        if (age >= 18) {
            eligibilityIfElse = "Eligible to vote";
        } else {
            eligibilityIfElse = "Not eligible to vote";
        }
        System.out.println("Using if-else: " + eligibilityIfElse);

        // Using ternary operator (shorthand)
        // Syntax: condition ? valueIfTrue : valueIfFalse
        String eligibilityTernary = (age >= 18) ? "Eligible to vote" : "Not eligible to vote";
        System.out.println("Using ternary: " + eligibilityTernary);

        // Another example: check if a number is even or odd.
        int number = 7;
        String evenOdd = (number % 2 == 0) ? "Even" : "Odd";
        System.out.println(number + " is " + evenOdd);
    }
}
```

Step 4.3: Run the Program

Save and run. Expected output:

text

Using if-else: Eligible to vote

Using ternary: Eligible to vote

7 is Odd

Step 4.4: Answer These Questions

1. What are the advantages of the ternary operator?
2. When should you avoid using the ternary operator?

✅ **Task 4 Complete**

- □ I created TernaryDemo.java
- □ I ran the program successfully
- □ I answered the questions

TASK 5: Analyzing Boolean Expressions

Objective

Analyze complex boolean expressions to determine all possible execution paths.

Step 5.1: Predict the Output

Before running any code, analyze the following expression and write down what you think the output will be.

java

```
boolean result = (5 > 3) && (4 < 2) || (10 == 10);

System.out.println(result);
```

Step-by-step analysis:

- 5 > 3 → true
- 4 < 2 → false
- (true) && (false) → false
- 10 == 10 → true
- false || true → true

Predicted output: true

Step 5.2: Verify by Writing Code

Create a file called BooleanAnalysis.java in the week3 folder.

java

```
// This program verifies the analysis of a complex boolean expression.

public class BooleanAnalysis {
    public static void main(String[] args) {

        // Evaluate the expression step by step and print intermediate results.
        boolean part1 = (5 > 3);         // true
        boolean part2 = (4 < 2);         // false
        boolean part3 = (10 == 10);      // true

        // Print each part.
        System.out.println("5 > 3: " + part1);
        System.out.println("4 < 2: " + part2);
        System.out.println("10 == 10: " + part3);

        // Combine using && and ||.
        // Operator precedence: && before ||
        // So: (part1 && part2) || part3
        boolean intermediate = part1 && part2; // true && false = false
        System.out.println("part1 && part2: " + intermediate);

        boolean finalResult = intermediate || part3; // false || true = true
        System.out.println("Final result: " + finalResult);

        // Direct evaluation for verification.
```

```
        boolean directResult = (5 > 3) && (4 < 2) || (10 == 10);
        System.out.println("Direct result: " + directResult);
    }
}
```

Run the program. Compare with your prediction.

Step 5.3: Analyze Another Expression

java

```
boolean result2 = !(5 > 3) || (4 < 2) && (10 == 10);
```

Analyze step by step:

- 5 > 3 → true
- !(true) → false
- 4 < 2 → false
- 10 == 10 → true
- false && true → false
- false || false → false

Predicted output: false

Write a program to verify this.

✅ **Task 5 Complete**

- □ I predicted the output correctly
- □ I wrote BooleanAnalysis.java and verified
- □ I analyzed the second expression

TASK 6: Evaluating Readability and Efficiency

Objective

Compare if-else-if and switch for the same logic, and evaluate which is better.

Step 6.1: Create Two Versions of a Program

Version A: Using if-else-if

Create a file called DayIfElse.java:

java

```
// This program prints the day name using if-else-if.


public class DayIfElse {
    public static void main(String[] args) {

        // Declare a variable for day number.
        int day = 3;

        // Use if-else-if to determine the day name.
        if (day == 1) {
            System.out.println("Monday");
        } else if (day == 2) {
            System.out.println("Tuesday");
        } else if (day == 3) {
            System.out.println("Wednesday");
        } else if (day == 4) {
            System.out.println("Thursday");
        } else if (day == 5) {
            System.out.println("Friday");
        } else if (day == 6) {
            System.out.println("Saturday");
        } else if (day == 7) {
```

```
            System.out.println("Sunday");
        } else {
            System.out.println("Invalid day");
        }
    }
}
```

Version B: Using switch

Create a file called DaySwitch.java:

java

```
// This program prints the day name using a switch statement.

public class DaySwitch {
    public static void main(String[] args) {

        // Declare a variable for day number.
        int day = 3;

        // Use switch to determine the day name.
        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
```

```
            System.out.println("Wednesday");
            break;
        case 4:
            System.out.println("Thursday");
            break;
        case 5:
            System.out.println("Friday");
            break;
        case 6:
            System.out.println("Saturday");
            break;
        case 7:
            System.out.println("Sunday");
            break;
        default:
            System.out.println("Invalid day");
            break;
    }
  }
}
```

Step 6.2: Compare and Answer

Run both programs. Then answer:

1. Which version is more readable? Why?
2. Which version is more efficient (faster)? (Hint: switch is often compiled to a jump table)
3. If you had 20 possible values, which would you prefer? Why?
4. If the conditions were ranges (e.g., marks >= 70), which would you use? Why?

✅ **Task 6 Complete**

- □ I created both files
- □ I ran both programs
- □ I answered the comparison questions

TASK 7: Debugging Faulty Conditionals

Objective

Debug a faulty conditional statement by tracing execution with various test inputs.

Step 7.1: Examine the Faulty Code

Create a file called FaultyConditional.java:

java

```
// This program has a logic error. Can you find it?


public class FaultyConditional {
    public static void main(String[] args) {


        // Declare a variable for the user's age.
        int age = 20;


        // The intention: check if age is between 18 and 65 (inclusive).
        // BUG: The condition is incorrect.
        if (age >= 18 || age <= 65) {
            System.out.println("You are of working age.");
        } else {
            System.out.println("You are not of working age.");
        }
```

```
    }
}
```

Step 7.2: Identify the Bug

Trace the code with these inputs:

- age = 20 → condition: 20 >= 18 is true, so true || ... is true → prints "You are of working age." (correct)
- age = 10 → condition: 10 >= 18 is false, 10 <= 65 is true → false || true is true → prints "You are of working age." (WRONG! Age 10 is not working age)
- age = 70 → condition: 70 >= 18 is true → true || ... is true → prints "You are of working age." (WRONG! Age 70 is not working age)

The bug: Used || instead of &&. The correct condition should be age >= 18 && age <= 65.

Step 7.3: Fix the Code

Create a corrected file called FixedConditional.java with the proper condition.

java

```
// This program corrects the logic error.


public class FixedConditional {
    public static void main(String[ ] args) {


        // Declare a variable for the user's age.
        int age = 20;


        // Correct condition: age must be >= 18 AND <= 65.
        if (age >= 18 && age <= 65) {
            System.out.println("You are of working age.");
        } else {
            System.out.println("You are not of working age.");
```

```
    }
  }
}
```

Step 7.4: Test with Multiple Inputs

Test with age = 10, age = 20, age = 70. Verify the corrected version works.

✅ **Task 7 Complete**

- □ I identified the bug
- □ I created FixedConditional.java
- □ I tested with multiple inputs

TASK 8: Push to GitHub

Step 8.1: Stage and Commit

1. Open the **Source Control** panel in VS Code.
2. Stage all new files.
3. Commit with message: Week 3: Conditionals - if-else, switch, ternary, debugging
4. Push to GitHub.

Step 8.2: Verify on GitHub

Go to your repository and confirm all Week 3 files are there.

✅ **Task 8 Complete**

- □ I committed and pushed all files
- □ I verified on GitHub

➢ **Final Self-Assessment Checklist**

- □ I can write if-else-if statements for multiple conditions
- □ I can use nested if-else for validation and decision-making
- □ I can use switch statements for menu-driven programs

- □ I can use the ternary operator for simple conditions
- □ I can analyze complex boolean expressions
- □ I can evaluate the readability and efficiency of different conditional structures
- □ I can debug faulty conditional statements
- □ I pushed all Week 3 code to GitHub

➢ **Troubleshooting Guide**

Problem	Solution
cannot find symbol: class Scanner	Add import java.util.Scanner;
InputMismatchException	You entered text instead of a number
fall-through in switch	Add break after each case
= instead of == in condition	Use == for comparison
&& vs || confusion	&& requires both true; || requires at least one true
Missing break in switch	Add break to prevent executing next case

➢ **Reflection Questions**

1. Which conditional structure did you find most useful? Why?
2. What was the most challenging part of this practical session?
3. What error message did you encounter, and how did you solve it?
4. In your own words, when would you use switch instead of if-else?
5. How will conditionals help you build real-world applications?

➢ What You've Achieved

You have just:

- Mastered **if-else-if** for multiple conditions
- Used **nested if-else** for validation
- Built a **menu-driven program** with switch
- Simplified code with the **ternary operator**
- **Analyzed** complex boolean expressions
- **Evaluated** different conditional structures
- **Debugged** a faulty conditional

You can now control the flow of your Java programs!

➢ Next Steps

In Week 4, you will:

1. Learn about **loops** (while, do-while, for)
2. Write programs that **repeat tasks**
3. Build a **number guessing game** and **pattern generator**
4. Continue pushing your code to GitHub

Additional Resources

Topic	Resource
if-else	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/if.html
switch	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/switch.html
Operators	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html

END OF PRACTICAL SESSION 3

"First, solve the problem. Then, write the code." – John Johnson

Instructor Note: You should complete this lab independently. I encourage experimentation with different values and conditions. The goal is to build intuition for when to use each conditional structure.